

JIT-AGENT: SCALING HARNESS INTELLIGENCE VIA JUST-IN-TIME HARNESS EVOLUTION

JIT-Agent Team

ABSTRACT

Agent capability is not determined by the model alone. The agent harness, encompassing memory management, planning strategy, action protocol, and tool/skill orchestration, can dominate the contribution of the underlying foundation model. Yet harness design remains manual, task-specific, and fundamentally unscalable. We present JIT-Agent, a harness intelligence model trained to synthesize task-adaptive agent harnesses on the fly for arbitrary off-the-shelf agentic LLMs. We formalize the agent harness as a composable, machine-generatable artifact governed by a fixed four-module protocol, and train JIT-Agent to *customize harnesses* for a given task at hand, *repair harnesses* for stable and reliable execution, and *self-evolve* by distilling performance signals from an expanding archive of prior harness configurations. Equipped with JIT-Agent as a harness helper, DeepSeek-V4-Flash surpasses GPT-5.6 on DeepSearchQA (+9.1) and OdysseyBench (+4.3), while the already strong GLM-5.2 gains up to +20.2 points. Across controlled evaluations, JIT-Agent-generated harnesses are performance-competitive with mature agent runtimes such as OpenCode and Claude Code and consistently improve multi-scale model families of DeepSeek V4, MIMO-V2.5, and Qwen3.6. To our knowledge, JIT-Agent is the first model purpose-built for just-in-time harness generation, establishing harness intelligence as a trainable, transferable, and compounding dimension of agent capability orthogonal to model scaling.

Code: <https://github.com/bingreeky/JIT>

1 INTRODUCTION

Agent harness matters. The capability of an LLM agent is jointly determined by two tightly coupled factors: the *foundation model* that produces reasoning and actions, and the *agent harness* that situates this model in a closed-loop execution environment (Meng et al., 2026b; Zhou et al., 2026; Ning et al., 2026). The harness decides what history is retained, how local intent is formed, which tools or skills are exposed, how actions are executed, and when verification or recovery is triggered (Meng et al., 2026b; Zhou et al., 2026). A strong model may fail when placed behind the wrong memory, planner, or action protocol; a strong harness, in turn, can only unlock capability when the model can understand and follow it (Lee et al., 2026c; Lin et al., 2026). Thus, agent intelligence is not a property of model weights alone, but of the model-harness pair (Lee et al., 2026c; Zhou et al., 2026). Harness design is therefore not an implementation detail, but a first-order determinant of agent performance (Meng et al., 2026b; Lin et al., 2026).

Ahead-of-Time (AOT) Harness This observation has motivated a growing line of work on test-time harness optimization (Lee et al., 2026c; Lin et al., 2026). Recent systems optimize harness code, prompts, tools, memories, skills, or control policies from trajectories and feedback, yielding strong improvements in coding, terminal-use, web interaction, and general agentic domains (Lee et al., 2026c; Lin et al., 2026). Despite their differences, many of these methods share an *Ahead-of-Time* (AOT) assumption: the harness is treated as a durable artifact to be optimized over an experience stream, with the hope that the resulting artifact will generalize across future tasks, domains, or model versions (Lee et al., 2026c; Lin et al., 2026). This is a powerful paradigm when the deployment distribution is stable and homogeneous (Lin et al., 2026; Zhang et al., 2025). However, it still asks the optimization loop to precompile a broadly useful harness before seeing the exact structure of each future problem (Lee et al., 2026c).

Just-in-Time (JIT) Harness. We instead turn to a *Just-in-Time* (JIT) view of harness construction. Different tasks plainly require different harness priors. Wide-search tasks may benefit

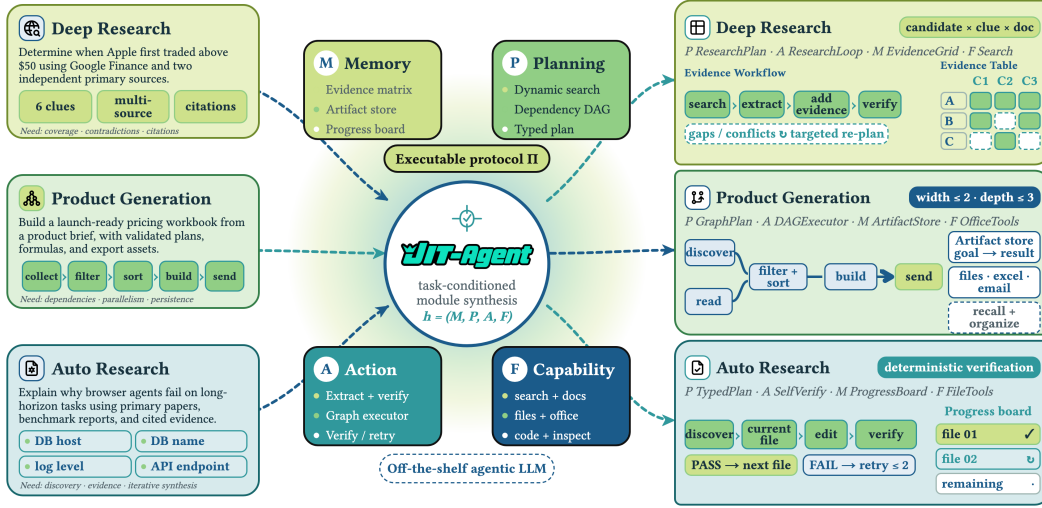


Figure 1: **Overview of JIT-Agent.** Given a task, JIT-Agent composes a problem-specific agent harness by instantiating (rather than simply combining) four modules: memory, planning, action, and capability. Different task structures therefore induce distinct executable protocols and state organizations, as illustrated by the specialized harnesses for deep research, product generation, and autonomous research.

from parallel evidence exploration (Qin et al., 2025); terminal tasks often favor a lean serial ReAct loop (Yao et al., 2023a; Merrill et al., 2026); deep-research tasks require working memory over retrieved evidence (Hu et al., 2025); and NL2Repo-style coding tasks are naturally mediated by a filesystem that stores patches, tests, traces, and repository state (Yang et al., 2024; Ning et al., 2026). In short, the appropriate harness is not only domain-dependent, but instance-dependent (Meng et al., 2026b; Zhou et al., 2026). Optimizing a single AOT harness across such heterogeneous demands is cumbersome: one must search a large design space, accumulate enough trajectories, and then hope that the resulting scaffold matches the next task (Lee et al., 2026c; Lin et al., 2026). We ask whether another possibility is viable: **Model-as-a-Harness**, where a trained meta-agent generates a task-specific harness on the spot, and an arbitrary agentic LLM executes under that harness.

JIT-Agent. We propose JIT-Agent, a compact meta-agent for just-in-time harness generation and evolution. At inference time, JIT-Agent receives a task specification, a protocol, an executable tool/skill registry, and a small retrieved context of prior harnesses, and emits an executable harness tailored to the task at hand. The resulting harness wraps an off-the-shelf agentic LLM for execution. As feedback and traces accumulate, JIT-Agent can revise the harness and update a harness archive, enabling test-time harness evolution while the generator itself remains fixed. Training such a system raises three challenges: ❶ **adaptivity**, namely how to match the generated harness to the task; ❷ **reliability**, namely how to ensure executable behavior and recover when synthesis fails (Pan et al., 2026; Ning et al., 2026); and ❸ **evolvability**, namely how to turn execution feedback into stronger future harnesses (Lee et al., 2026c; Lin et al., 2026). Together, these three requirements define what we call **harness intelligence**:

Harness Intelligence

Harness intelligence is the capacity to construct and refine the operational scaffold through which a model acts. It has three defining properties: **adaptivity**, matching the harness to the task and backbone; **reliability**, producing executable behavior and recovering when synthesis fails; and **evolvability**, turning execution feedback into stronger future harnesses. Intelligence about the harness begins where competence within a fixed harness ends.

JIT-Agent realizes harness intelligence through a three-stage training recipe over a protocol-induced harness space (Meng et al., 2026b; Zhou et al., 2026). First, we factor an agent harness into a four-tuple of *memory*, *planning*, *action*, and *capability-orchestration* modules, and build HARNESSFACTORY, a unified codebase of representative scaffolds under this common interface (Hu et al., 2025; Erdogan

et al., 2025a; Yao et al., 2023a; Xu & Yan, 2026). Following the view that *code carries the harness* (Ning et al., 2026; Pan et al., 2026), this representation makes harness generation feasible: JIT-Agent generates structured executable modules rather than unconstrained agent programs. ❶ **Stage I** then teaches task-conditioned customization from teacher-generated protocol-compliant examples, giving the generator adaptivity over the shared design space. Second, ❷ **Stage II** converts failed generations into bounded repair trajectories, turning compiler errors, interface mismatches, and runtime failures into supervision for reliable recovery. Third, ❸ **Stage III** introduces *Evolutionary Group-Decoupled Policy Optimization* (Evo-GDPO), which optimizes JIT-Agent to propose harnesses that overtake the current archive frontier while separately normalizing reward, latency, and cost. This supplies evolvability by turning test-time harness improvement into a trained capability rather than an external search heuristic. Through this pipeline, we train **JIT-Agent-27B**, a **harness intelligence model** that can customize, repair, and evolve task-specific harnesses, shifting harness engineering from AOT artifacts to native JIT synthesis.

Experiment Findings. Across nine benchmarks, JIT-Agent consistently strengthens its underlying backbones: DeepSeek-V4-Flash surpasses GPT-5.6 by 9.1 points on DeepSearchQA and 4.3 on OdysseyBench, while GLM-5.2 gains up to 20.2 points. The generated harnesses also remain competitive with mature runtimes and transfer across DeepSeek V4, MIMO-V2.5, and Qwen3.6.

Contributions. (I) We formulate *Model-as-a-Harness* and define *harness intelligence* as the learned ability to construct, repair, and evolve an agent’s operational scaffold. (II) We instantiate this paradigm with JIT-Agent, which combines a four-module executable protocol, HARNESSFACTORY, and a three-stage customization–repair–evolution pipeline, and (III) validate that JIT-generated harnesses improve diverse backbones while remaining competitive with strong production harnesses.

2 RELATED WORK

Harness Engineering and Modularization. Harnesses have diversified across coding and general-purpose runtimes such as Claude Code, Codex, Hermes, and OpenClaw (Anthropics, 2025; OpenAI, 2025; Nous Research, 2026; OpenClaw Contributors, 2026), embodied systems (Zhang et al., 2026c; Lee et al., 2026b), search agents (Apodex Team, 2026; Qin et al., 2025), and memory-centric systems (Chhikara et al., 2025; Hu et al., 2026a; Zhang et al., 2025). Despite domain-specific implementations, they share an operational layer that governs memory, planning and control, action verification, and tool/skill orchestration, making capability a property of the model–harness pair (Meng et al., 2026b; Zhou et al., 2026; Pan et al., 2026). Modular representations expose these choices through explicit interfaces, allowing components to be substituted and optimized independently (Chen et al., 2026b; Ning et al., 2026). We therefore use the compact executable factorization $\mathbf{h} = (\mathbf{M}, \mathbf{P}, \mathbf{A}, \mathbf{F})$ for memory, planning, action, and capability orchestration.

Harness Optimization. With explicit interfaces, harness construction becomes optimization over executable artifacts rather than a fixed engineering choice (Khatab et al., 2023; Yuksekgonul et al., 2024; Hu et al., 2024b; Shang et al., 2024). Recent methods extend optimization from prompts and workflow graphs to complete harness code, commonly using a diagnose–propose–validate loop in which traces guide edits and validation determines retention (Zhang et al., 2024; Lee et al., 2026c; Lin et al., 2026; Chen et al., 2026b). Recursive, hierarchical, and online variants improve this loop, but still primarily optimize a persistent artifact against accumulated experience and are therefore ahead-of-time relative to a new task instance (Lee et al., 2026a; Zhou, 2026; Nie et al., 2026; Shao et al., 2026). JIT-Agent instead amortizes search into a trained generator that constructs a task-conditioned harness just in time; Table 4 summarizes this distinction.

3 UNIFIED HARNESS CODEBASE

3.1 MODULARIZED HARNESS DESIGN SPACE

An agent harness is the operational layer that turns a foundation model into a closed-loop agent (Meng et al., 2026a; Zhou et al., 2026): it determines what from prior interaction is retained (Hu et al., 2026b), how intermediate directives are formed (Cao et al., 2025), which external capabilities are exposed at each stage (Xu & Yan, 2026; Yang et al., 2026), and how control advances. Harness generation is therefore defined over programs rather than unconstrained text. We distinguish the raw generation space $\mathcal{G}$, syntactically valid harnesses $\mathcal{H}^{\text{syn}}$, protocol-compliant harnesses $\mathcal{H}_{\Pi}$, and their executable subset $\mathcal{H}_{\Pi}^{\text{exec}}$, with $\mathcal{G} \supseteq \mathcal{H}^{\text{syn}} \supseteq \mathcal{H}_{\Pi} \supseteq \mathcal{H}_{\Pi}^{\text{exec}}$. The fixed protocol Π specifies

module schemas and interfaces, their lifecycle, validation rules, and the shared execution semantics. It removes incidental variation in language and runtime while retaining the principal operational choices that distinguish existing harnesses (Wu et al., 2023; Microsoft, 2024; LangGraph, 2025; Luo et al., 2025). JIT-Agent makes these choices explicit through four interoperable modules: *how history is compressed*, *how local intent is formed*, *how tools and skills are orchestrated*, and *how control advances*. Let τ be a task, π_ψ a frozen backbone executor, and $\mathcal{C}_\tau$ the capability registry available (e.g., tools, APIs, skills) to that task. Running a harness $\mathbf{h}$ with π_ψ induces a closed-loop trajectory

$$\xi \sim \text{Rollout}(\tau, \pi_\psi, \mathbf{h}, \mathcal{C}_\tau; \Pi) = (\mathbf{s}_1, e_1, \mathbf{o}_1, \dots, \mathbf{s}_T, e_T, \mathbf{o}_T), \quad (1)$$

where $\mathbf{s}_t \in \mathcal{S}$ is the maintained controller state, $e_t \in \mathcal{A} = \mathcal{U} \sqcup \mathcal{Y}$ is either an executable call from $\mathcal{U}$ or a terminal output from $\mathcal{Y}$, $\mathbf{o}_t \in \mathcal{O}$ is the resulting observation, and T is the protocol- or budget-bounded stopping time. Our key assumption is that every $\mathbf{h} \in \mathcal{H}_\Pi$ admits:

$$\mathbf{h} = (\mathbf{M}, \mathbf{P}, \mathbf{A}, \mathbf{F}) \in \mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}, \quad (2)$$

where $\mathbf{M}$, $\mathbf{P}$, $\mathbf{A}$, and $\mathbf{F}$ denote memory, planning, action, and capability-orchestration modules, and $\mathfrak{M}$, $\mathfrak{P}$, $\mathfrak{A}$, and $\mathfrak{F}$ are their protocol-compatible implementation spaces. The tuple follows this conceptual decomposition; at runtime, its dependency order is $\mathbf{M} \rightarrow \mathbf{P} \rightarrow \mathbf{F} \rightarrow \mathbf{A}$. All modules operate against the same frozen π_ψ , whose common runtime dependence is suppressed below. The protocol maintains both the immutable event history $\xi_{<t}$ and the mutable controller state $\mathbf{s}_t$, which interact as

$$\begin{aligned} \mathbf{v}_t &= \mathbf{M}(\xi_{<t}, \mathbf{s}_t) \in \mathcal{V}, & \text{history} &\rightarrow \text{view}, \\ \mathbf{d}_t &= \mathbf{P}(\tau, \mathbf{s}_t, \mathbf{v}_t) \in \mathcal{D}_{\text{dir}}, & \text{view} &\rightarrow \text{local directive}, \\ \mathcal{C}_t &= \mathbf{F}(\mathcal{C}_\tau, \mathbf{s}_t, \mathbf{v}_t, \mathbf{d}_t) \subseteq \mathcal{C}_\tau, & \text{directive-conditioned capability orchestration}, \\ (\mathbf{s}_{t+1}, e_t) &= \mathbf{A}(\mathbf{s}_t, \tau, \mathbf{v}_t, \mathbf{d}_t, \mathcal{C}_t) \in \mathcal{S} \times \mathcal{A}, & \text{control update and action emission}, \end{aligned} \quad (3)$$

where $\mathcal{V}$ and $\mathcal{D}_{\text{dir}}$ are the view and directive spaces. The registry $\mathcal{C}_\tau$ contains callable tools (e.g., bash tools, APIs, and MCPs (Sarkar & Sarkar, 2025)) as well as higher-level agent skills (Xu & Yan, 2026; ClaudeCode, 2025); a harness without an explicit planner remains type-consistent through the null directive $\mathbf{d}_\emptyset \in \mathcal{D}_{\text{dir}}$ returned by $\mathbf{P}_\emptyset$. Thus, $\mathbf{M}$ constructs a view of realized history, $\mathbf{P}$ forms a local directive, $\mathbf{F}$ activates relevant capabilities, and $\mathbf{A}$ updates the controller state and emits the next action. The shared kernel $\text{Exec} : \mathcal{U} \times 2^{\mathcal{C}_\tau} \rightarrow \mathcal{O}$ sets $\mathbf{o}_t = \text{Exec}(e_t; \mathcal{C}_t)$ for $e_t \in \mathcal{U}$ and $\mathbf{o}_t = \perp$ for $e_t \in \mathcal{Y}$, where $\perp \in \mathcal{O}$ is the terminal null observation; it then updates $\xi_{\leq t} = \xi_{<t} \oplus (\mathbf{s}_t, e_t, \mathbf{o}_t)$, with $\oplus$ denoting event append. Execution starts from $\xi_{<1} = \emptyset$ and a protocol-defined $\mathbf{s}_1 \in \mathcal{S}$, and terminates when $e_t \in \mathcal{Y}$, yielding $y = e_T \in \mathcal{Y}$. This factorization maps otherwise heterogeneous programs into comparable coordinates in $\mathfrak{M} \times \mathfrak{P} \times \mathfrak{A} \times \mathfrak{F}$: canonical ReAct uses full-history memory, no explicit planner, a ReAct loop, and the full capability registry; engineered variants such as Codex and OpenCode add compact memory and todo-based planning (OpenAI, 2025; OpenCode-AI, 2025); and recursive systems such as ROMA, AOrchestra, and RLM instantiate subproblem memory, decomposition, recursive action, and routed capabilities (Alzu'bi et al., 2026; Ruan et al., 2026; Zhang et al., 2026a). Given this modularized design space, we next introduce HARNESSFACTORY, whose role is to test the expressiveness of $\mathcal{H}_\Pi$ and furnish diverse source material for JIT-Agent's meta-harness design.

3.2 HARNESSFACTORY

Under the shared protocol Π and a common kernel, HARNESSFACTORY re-implements 13 representative scaffolds spanning memory, planning, aggregation, hierarchical control, and recursive execution; Table 1 gives the full inventory. These implementations form the protocol-compatible seed bank $\mathcal{B}_0$ of size $K_0 = 13$. The bank serves a dual role: its seeds anchor Stage-I synthesis, while later states supply the prior population against which new designs are evaluated. After n evolution rounds, $\mathcal{B}_0$ grows into $\mathcal{B}_n \supseteq \mathcal{B}_0$, whose entries associate each retained harness with its task and observed reward, latency, and cost; this evolution is formalized in Section 4.1.

4 METHOD

4.1 TRAINING PIPELINE

Training follows the inference-time lifecycle of JIT-Agent: synthesize a task-conditioned harness, recover unstable generations, and then improve from stronger archive states online. Let $\tau \sim \mathcal{D}_{\text{task}}$

Table 1: Seed bank $\mathcal{B}_0$: 13 hand-written harnesses instantiating the four-module protocol Π . Each row is a complete harness; columns follow the conceptual order of memory, planning, action, and capability orchestration.

Harness	♣ Memory $M \in \mathfrak{M}$	♦ Planning $P \in \mathfrak{P}$	♥ Action $A \in \mathfrak{A}$	♠ Capability Orchestration $F \in \mathfrak{F}$
REACT [69]	FullHistory	No explicit planner	ReAct	Full registry
PLAN-AND-EXECUTE [14]	FullHistory	Linear roadmap	ReAct	Full registry
RESUM [63]	ReSum memory	No explicit planner	ReAct	Full registry
FLASH-SEARCHER [51]	FullHistory	DAG planning	ReAct	Full registry
GAM [65]	GAM retrieval	DAG	ReAct	Full registry
MEMOBRAIN [50]	Reasoning graph	No explicit planner	Marker-guided execution	Full registry
AGGAGENT [29]	Isolated rollout histories	No explicit planner	Multi-rollout aggregation	Full registry
OAGENT [80]	Coordinator history	No explicit planner	Ensemble voting	Full registry
AGENTFOLD [70]	AgentFold memory	DAG	ReActFold	Full registry
HIAGENT [20]	Hierarchical memory	No explicit planner	ReAct	Full registry
DEEPAGENT [30]	Three-tier memory	No explicit planner	Marker-guided execution	Tool search
ROMA [1]	Context isolation	Atomizer + DAG	Recursive execution	Full registry
AORCHESTRA [53]	Context isolation	Atomizer + DAG	Recursive execution	Agent Delegation

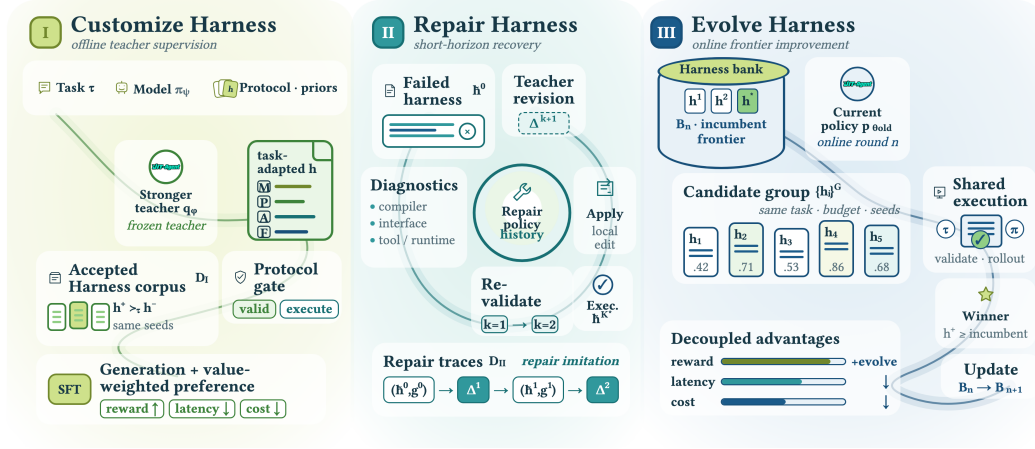


Figure 2: **Training pipeline of JIT-Agent.** Stage I learns task-conditioned harness customization. Stage II converts failed harnesses and execution diagnostics into bounded repair trajectories. Stage III performs online evolution by comparing candidate harnesses against the incumbent bank, optimizing decoupled reward, latency, and cost advantages, and retaining frontier-improving designs.

denote a training task, $\mathcal{C}_\tau$ its capability registry, and $\mathcal{E}_\tau$ a small reference context from the harness bank. The generation context is $\mathbf{c}_\tau = (\tau, \Pi, \mathcal{C}_\tau, \mathcal{E}_\tau)$, from which p_θ generates a harness $\mathbf{h} \in \mathcal{G}$. A frozen executor $\pi_\psi \sim \mathcal{M}$ evaluates task reward, latency, and monetary cost, while $\text{Valid}_\Pi(\mathbf{h}; \tau, \pi_\psi, \mathcal{C}_\tau) \in \{0, 1\}$ checks protocol and execution validity and returns structured diagnostics upon failure. The three stages optimize this generator through offline supervision, repair-trajectory learning, and online policy improvement, respectively.

Stage I: Data Preparation. Stage I uses a frozen, stronger teacher q_ϕ to synthesize task-adapted harnesses under the fixed four-module protocol. For each task, $\mathcal{E}_\tau$ contains three scaffolds sampled from the task-type-matched subset $\mathcal{B}_0^{(d(\tau))}$, where $d(\tau)$ denotes the task type. The teacher receives the resulting context $\mathbf{c}_\tau$, and its generation is retained only after protocol validation and execution checks. Training tasks come from existing agentic benchmarks and environments (Wu et al., 2025; Shi et al., 2025; Bai et al., 2026; Song et al., 2026), augmented with synthesized tasks.

Stage I: Training Setup. We first apply standard supervised fine-tuning to accepted teacher generations, teaching JIT-Agent to map a task and its reference context directly to a protocol-compliant harness. Since executable harnesses can still differ substantially in quality and efficiency, we then compare candidates under the same backbone and evaluation seeds. A preference is retained

only when reward improves, neither efficiency axis degrades, and at least one efficiency gain is strict:

$$\begin{aligned} \mathbf{h}^+ \succ_{\tau} \mathbf{h}^- &\iff r^+ > r^- \wedge \ell^+ \leq \ell^- \wedge \kappa^+ \leq \kappa^- \wedge (\ell^+ < \ell^- \vee \kappa^+ < \kappa^-), \\ \Delta_{\text{val}}(\tau; \mathbf{h}^+, \mathbf{h}^-) &= \alpha_r(r^+ - r^-) + \alpha_{\ell}[\ell^- - \ell^+]_+ + \alpha_{\kappa}[\kappa^- - \kappa^+]_+, \end{aligned} \quad (4)$$

where $(r^{\pm}, \ell^{\pm}, \kappa^{\pm})$ are repeated-rollout averages for $(\mathbf{h}^+, \mathbf{h}^-)$, $[x]_+ = \max(x, 0)$, and $\alpha_r, \alpha_{\ell}, \alpha_{\kappa} \geq 0$ weight the reward, latency, and cost gaps. The resulting pairs supervise a standard DPO objective, weighted by Δ_{val} and anchored to the frozen Stage-I SFT checkpoint. Combined with generation imitation, this objective preserves protocol-valid structure while favoring harnesses that are both more effective and more efficient.

Stage II: Data Preparation. Stage I optimizes for protocol compliance, but some generated harnesses still fail static or runtime validation. Rather than discarding them, Stage II pairs each failed harness $\tilde{\mathbf{h}}^{(0)}$ with a structured diagnostic $\mathbf{g}^{(0)}$ that records compiler errors, interface mismatches, tool-call failures, or runtime exceptions. At repair round k , the teacher proposes a structured revision $\Delta^{(k+1)}$ from patch space $\mathcal{P}$; Apply updates the harness deterministically, after which validation produces the next diagnostic. We retain only trajectories that become executable within two rounds:

$$\begin{aligned} \tilde{\mathbf{h}}^{(k+1)} &= \text{Apply}(\tilde{\mathbf{h}}^{(k)}, \Delta^{(k+1)}), \quad k \in \{0, 1\}, \\ K^* &= \min \left\{ k \in \{1, 2\} : \text{Valid}_{\Pi}(\tilde{\mathbf{h}}^{(k)}; \tau, \pi_{\psi}, \mathcal{C}_{\tau}) = 1 \right\}. \end{aligned} \quad (5)$$

Supervision is constructed only when K^* exists, focusing the corpus on realistic, locally recoverable failures rather than those requiring wholesale redesign.

Stage II: Training Setup. Stage II trains JIT-Agent by teacher-forced imitation of the successful revisions $\{\Delta^{(k+1)}\}_{k=0}^{K^*-1}$. Each revision is conditioned on $\mathbf{c}_{\tau}$ and the complete preceding harness–diagnostic history $\{(\mathbf{h}^{(j)}, \mathbf{g}^{(j)})\}_{j=0}^k$. Because $K^* \leq 2$, the model learns the short-horizon regime needed at deployment: use execution feedback to make a small number of high-leverage revisions that restore protocol-valid execution.

Stage III: Data Preparation. Stage III treats *test-time harness evolution* as a trainable capability: JIT-Agent learns to propose harnesses that surpass prior designs and then become stronger references for subsequent tasks. We term the resulting online objective **Evolutionary Group-Decoupled Policy Optimization (Evo-GDPO)**, inspired by (Liu et al., 2026a). At online round n , we sample a task τ , retrieve prior high-quality designs $\mathcal{E}_{\tau,n}$ from the current bank $\mathcal{B}_n$, and form $\mathbf{c}_{\tau,n} = (\tau, \Pi, \mathcal{C}_{\tau}, \mathcal{E}_{\tau,n})$. The model samples $G > 1$ candidates independently from the current rollout policy; candidates and retrieved designs are evaluated under the same frozen executor $\pi_{\psi} \sim \mathcal{M}$, budget, and seeds. The highest-reward retrieved harness, with ties broken by lower latency and then lower cost, supplies the incumbent statistics $(b_r, b_{\ell}, b_{\kappa})$.

Stage III: Training Setup. Every candidate is validated before execution; one that remains invalid after bounded repair receives the minimum task reward, while its repair latency and cost remain part of the measured efficiency. For executable candidate i , let r_i , $\bar{\ell}_i$, and $\bar{\kappa}_i$ denote reward, mean latency, and mean monetary cost over repeated rollouts. The reward channel is primary, whereas efficiency gains activate only when incumbent reward is preserved:

$$R_i^{\text{rew}} = r_i + \lambda_{\text{evo}} [r_i - b_r]_+, \quad R_i^{\text{lat}} = \mathbb{I}[r_i \geq b_r] [b_{\ell} - \bar{\ell}_i]_+, \quad R_i^{\text{cost}} = \mathbb{I}[r_i \geq b_r] [b_{\kappa} - \bar{\kappa}_i]_+, \quad (6)$$

where $\lambda_{\text{evo}} \geq 0$ controls the improvement bonus and $\mathbb{I}[\cdot]$ is the indicator function. Evo-GDPO normalizes the three channels independently within each candidate group before aggregation, preventing their scales from overwhelming one another:

$$\begin{aligned} A_i^m &= \frac{R_i^m - \text{mean}(\{R_j^m\}_{j=1}^G)}{\text{std}(\{R_j^m\}_{j=1}^G) + \varepsilon_{\text{num}}}, \quad m \in \{\text{rew}, \text{lat}, \text{cost}\}, \\ A_i^{\Sigma} &= w_{\text{rew}} A_i^{\text{rew}} + w_{\text{lat}} A_i^{\text{lat}} + w_{\text{cost}} A_i^{\text{cost}}, \quad w_{\text{rew}} > w_{\text{lat}} + w_{\text{cost}}. \end{aligned} \quad (7)$$

The nonnegative weights sum to one, and the stated inequality makes task reward dominant; $\varepsilon_{\text{num}} > 0$ stabilizes normalization. We further normalize A_i^{Σ} across the training batch and use it in a standard clipped PPO objective with a token-level KL penalty to the frozen Stage-II checkpoint. Thus, unlike

standard group-relative optimization, Evo-GDPO rewards candidates for overtaking prior designs and, when reward is preserved, for doing so more efficiently. After optimization, we update the harness bank conservatively: a candidate is retained only if it matches or exceeds the current reward frontier and then strictly improves at least one frontier dimension—either reward itself, latency, or cost. During training, this feedback updates both θ and $\mathcal{B}_n$; at deployment, θ remains frozen.

4.2 INFERENCE ARCHITECTURE

JIT-Agent supports two inference modes: *static inference* and *streaming inference*; they differ in whether experience is discarded after the task or retained to support subsequent tasks.

Static inference. In this mode, we introduce a lightweight form of test-time scaling: JIT-Agent generates N harnesses in parallel, selects one of them, and executes only the selected harness. This increases candidate diversity without increasing the number of environment rollouts. The selected harness follows the same validation and bounded-repair procedure described above.

Streaming inference. Streaming inference mode is designed to carry useful experience forward across a sequence of tasks. For the n -th task τ_n , JIT-Agent retrieves from the current bank $\mathcal{B}_n$, generates and selects a harness $\mathbf{h}_n^\dagger$, and executes it once. The resulting environment feedback is then used only to determine whether this experience should update the bank:

$$\begin{aligned} \xi_n &\sim \text{Rollout}(\tau_n, \pi_\psi, \mathbf{h}_n^\dagger, \mathcal{C}_{\tau_n}; \Pi), \mathbf{m}_n = \text{Eval}(\xi_n), \\ \mathcal{B}_{n+1} &= \text{Update}_{\text{III}}(\mathcal{B}_n; \tau_n, \mathbf{h}_n^\dagger, \mathbf{m}_n), \mathcal{E}_{\tau_{n+1}, n+1} = \text{Retrieve}(\tau_{n+1}; \mathcal{B}_{n+1}), \end{aligned} \quad (8)$$

where $\mathbf{m}_n = (r_n, \bar{\ell}_n, \bar{\kappa}_n)$ contains reward, mean latency, and mean monetary cost. Following the Stage-III retention rule, $\text{Update}_{\text{III}}$ leaves $\mathcal{B}_n$ unchanged when the completed harness provides no admissible improvement. Otherwise, the retained harness becomes a potential reference for later tasks. Streaming inference therefore transfers prior experience through the evolving harness bank, without injecting environment feedback into the current rollout or updating model parameters.

5 EXPERIMENTS AND ANALYSIS

5.1 EXPERIMENT SETUP

Evaluation Benchmarks. We evaluate JIT-Agent on nine benchmarks spanning four task types. **Deep Research** includes BrowseComp-Plus (Chen et al., 2025b) (accuracy), DeepSearchQA (Gupta et al., 2026) (answer F1), and xBench-DeepSearch (xBench-DS) (Chen et al., 2025a) (accuracy); **Daily Work** includes AgentIF-Oneday (Chen et al., 2026a) (normalized weighted rubric score) and PinchBench (PinchBench, 2026) (average score); **Planning** includes the DeepPlanning-Shopping (cart match rate) and DeepPlanning-Travel (composite constraint score) subsets (Zhang et al., 2026b); and **Workspace** includes OfficeBench (Wang et al., 2024) and OdysseyBench (Wang et al., 2025) (task success rate for both). Together, these benchmarks cover evidence-intensive search, long-horizon instruction following, constraint-aware planning, and multi-application workspace execution. We report all metrics on a 0–100 scale; higher is better.

Backbone Models. JIT-Agent is trained based on Qwen3.6-27B. We primarily instantiate JIT-Agent with GLM-5.2 (Z.ai, 2026) and DeepSeek-V4-Flash-Preview (DeepSeek-AI, 2026). These backbones provide complementary operating points: GLM-5.2 is a strong open long-horizon model, whereas DeepSeek-V4-Flash emphasizes inference efficiency. To test the generalization, additional models from Qwen3.6, MIMO-V2.5 are also used.

Baselines. We use two complementary baseline groups. First, the broad comparison includes vanilla **Qwen3.7-Plus** (Qwen Team, 2026), **GLM-5.2**, **DeepSeek-V4-Flash/Pro** (both preview version), **Kimi K2.7 Code** (Moonshot AI, 2026), **GPT-5.6-Sol** (OpenAI, 2026), and **Gemini 3.1 Pro/3.5 Flash** (Google DeepMind, 2026a;b). Second, holding the backbone fixed, we compare JIT-Agent with five reusable agent harnesses: **CLAUDE CODE** (Anthropics, 2025), **CODEX** (OpenAI, 2025), **OPENCODE** (OpenCode-AI, 2025), **HERMES** (Nous Research, 2026), and **NANOBOT** (HKUDS, 2026). The first group measures end-to-end competitiveness, while the second isolates the contribution of harness design from that of model weights.

5.2 MAIN RESULTS

Table 2 compares JIT-generated harnesses with vanilla backbones and frontier models across nine benchmarks. Replacing the default scaffold improves all 18 matched backbone–benchmark pairs:

Table 2: Main results across nine agentic benchmarks. Rows prefixed by JIT-Agent use the same backbone as their vanilla counterpart but replace its default scaffold with a JIT-generated harness. Scores are reported on a 0–100 scale; higher is better. Top-1, Top-2, and Top-3 results in each column are highlighted in soft green, yellow-green, and yellow, respectively; Top-1 is also bolded.

Model	Deep Research			Daily Work		Planning		Workspace	
	BC+	DSQA	xBench	AgentIF	PinchBench	Shop	Travel	Office	Odyssey
Qwen3.7-Plus	70.5	78.0	75.0	59.9	80.5	75.2	52.1	56.6	67.7
GLM-5.2	72.0	89.2	76.0	63.0	87.0	78.2	62.8	63.0	75.3
DeepSeek-V4-Flash	68.1	76.2	70.1	58.4	81.7	59.1	54.8	61.0	71.0
DeepSeek-V4-Pro	71.4	72.4	79.0	56.5	61.1	71.1	55.2	62.0	72.0
Kimi K2.7 Code	73.6	87.8	79.0	57.3	76.1	72.8	56.9	65.8	69.9
GPT-5.6	76.9	76.0	81.0	68.0	84.2	83.7	84.9	65.3	68.7
Gemini 3.1 Pro	67.5	75.8	83.0	60.1	81.0	77.4	70.7	60.6	74.0
Gemini 3.5 Flash	75.0	88.0	85.0	64.0	74.2	76.2	50.3	63.3	78.0
JIT-Agent + GLM-5.2	78.0	93.9	88.0	69.9	93.3	83.4	83.0	68.4	78.7
JIT-Agent + DeepSeek-V4-Flash	74.0	85.1	82.0	63.8	92.9	83.9	61.3	63.4	73.0

Table 3: Controlled comparison with advanced agent harnesses. For each fixed backbone, we report task performance (Perf.), average token consumption per case in thousands (#Tokens (K)), and API cost per case in USD. Performance is better when higher; tokens and cost are better when lower. The best value within each backbone group is bolded.

Backbone	Harness	DeepSearchQA			xBench-DS			AgentIF		
		Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓	Perf. ↑	#Tokens (K)	Cost ↓
DeepSeek-V4-Flash	CLAUDE CODE	79.6	625	\$0.088	75.0	559	\$0.079	66.9	808	\$0.114
	CODEX	77.8	760	\$0.107	70.0	680	\$0.096	58.5	870	\$0.123
	OPENCODE	75.9	1,832	\$0.258	65.0	1,157	\$0.159	48.1	950	\$0.135
	HERMES	69.9	1,157	\$0.163	72.0	1,254	\$0.177	60.3	1,000	\$0.142
	NANOBOT	80.4	924	\$0.131	78.0	527	\$0.075	53.1	1,034	\$0.147
	JIT-Agent	85.1	400	\$0.066	82.0	212	\$0.039	63.8	476	\$0.097
Qwen3.6-Flash	CLAUDE CODE	72.8	710	\$0.140	58.0	650	\$0.128	55.4	900	\$0.177
	CODEX	68.5	980	\$0.193	52.0	874	\$0.172	34.2	839	\$0.170
	OPENCODE	64.1	1,968	\$0.384	44.0	1,300	\$0.256	38.7	961	\$0.217
	HERMES	60.8	1,319	\$0.256	36.0	1,155	\$0.227	49.7	1,184	\$0.223
	NANOBOT	74.2	892	\$0.197	63.0	597	\$0.119	43.5	950	\$0.187
	JIT-Agent	70.3	464	\$0.095	70.0	300	\$0.069	58.3	394	\$0.078

the average score rises from 74.1 to 81.8 on GLM-5.2 (+7.7) and from 66.7 to 75.5 on DeepSeek-V4-Flash (+8.8). The largest gains appear on long-horizon planning, with +24.8 on DeepPlanning-Shopping for DeepSeek-V4-Flash (59.1 → 83.9) and +20.2 on DeepPlanning-Travel for GLM-5.2 (62.8 → 83.0); DeepSeek-V4-Flash also gains 11.9 on xBench-DS and 8.9 on DeepSearchQA. Despite using open backbones, the JIT-equipped systems lead eight of nine columns: JIT-Agent + GLM-5.2 ranks first on seven benchmarks, including 93.9 on DeepSearchQA, 69.9 on AgentIF, and 93.3 on PinchBench, while JIT-Agent + DeepSeek-V4-Flash leads DeepPlanning-Shopping at 83.9 and exceeds DeepSeek-V4-Pro on every reported benchmark by 8.7 points on average. DeepPlanning-Travel is the sole exception, where the JIT-equipped GLM-5.2 remains within 1.9 points of GPT-5.6. These results show that task-adaptive harnesses consistently raise the capability of a fixed backbone rather than merely benefiting a small subset of tasks.

5.3 COMPARISON WITH ADVANCED AGENT HARNESSES

Table 3 controls the backbone and compares six harnesses on DeepSeek-V4-Flash and Qwen3.6-Flash across DeepSearchQA, xBench-DS, and AgentIF, reporting performance, tokens, and API cost per case. JIT-Agent achieves the best performance in four of six settings: it exceeds the strongest fixed harness by 4.7 points on DeepSearchQA and 4.0 on xBench-DS with DeepSeek-V4-Flash, and by 7.0 on xBench-DS and 2.9 on AgentIF with Qwen3.6-Flash. In the two remaining settings, it trails the best score by only 3.1 and 3.9 points while using substantially fewer tokens, confirming that no fixed harness dominates across tasks. More importantly, JIT-Agent has the lowest token consumption and

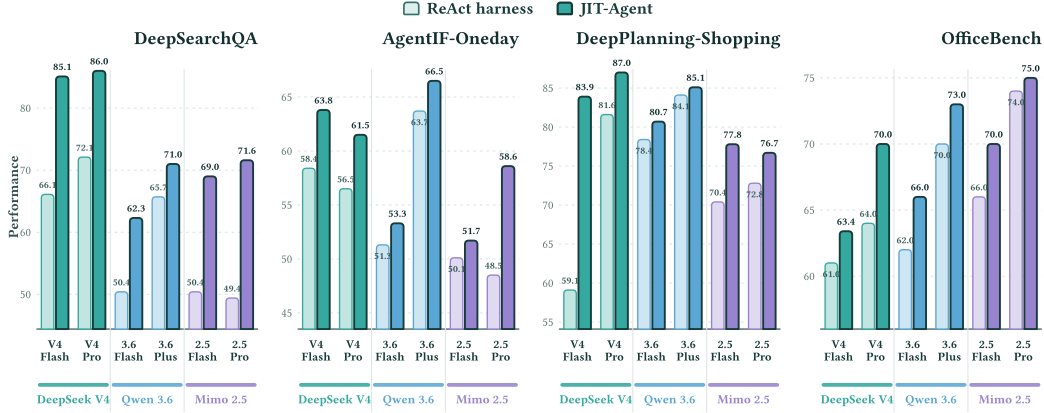


Figure 4: **JIT-generated harnesses consistently improve paired backbones over ReAct.** The four panels report DeepSearchQA, AgentIF-Oneday, DeepPlanning-Shopping, and OfficeBench. Within each panel, paired bars compare the same backbone under a fixed ReAct harness and a JIT-generated harness, with six backbones grouped by model family and scores shown above the bars. DeepSearchQA uses a 100-example subset; the other three benchmarks use 50-example subsets.

cost in all six settings, reducing per-case cost by 14.9–54.1% relative to the cheapest fixed alternative, or 36.0% on average. On DeepSeek-V4-Flash xBench-DS, it cuts tokens from 527K to 212K and cost from \$0.075 to \$0.039 while raising performance from 78.0 to 82.0; on Qwen3.6-Flash AgentIF, it reaches 58.3 with 394K tokens at \$0.078, compared with at least 839K tokens and \$0.170 for fixed harnesses. The gains therefore arise from task-conditioned scaffolds rather than longer trajectories.

5.4 COST-PERFORMANCE PARETO FRONTIERS

Figure 3 visualizes the cost–performance geometry of the controlled comparison on DeepSearchQA and AgentIF. Each point is a backbone–harness pair from Table 3; points closer to the upper-left corner are preferable. On DeepSearchQA, DeepSeek-V4-Flash + JIT-Agent is a strong Pareto-optimal operating point among the evaluated pairings, scoring 85.1 at a per-case cost of \$0.066. Relative to the strongest fixed harness, NanoBot, it gains 4.7 points while reducing cost by 49.6% (\$0.131 → \$0.066). For Qwen3.6-Flash, the generated harness provides a lower-compute operating point: it scores 70.3 at \$0.095, compared with NanoBot’s 74.2 at \$0.197, cutting cost by 51.8% for a 3.9-point trade-off. AgentIF exhibits a different but complementary frontier. With DeepSeek-V4-Flash, JIT-Agent reduces cost by 14.9% relative to Claude Code (\$0.097 vs. \$0.114) while remaining within 3.1 points of its performance (63.8 vs. 66.9); both therefore lie on the frontier.

With Qwen3.6-Flash, JIT-Agent strictly dominates all fixed harnesses, improving the best score from 55.4 to 58.3 while lowering the minimum observed cost from \$0.170 to \$0.078. The frontier is therefore task-dependent, but the consistent leftward shift of the JIT points shows that task-adaptive harnesses improve inference efficiency rather than purchasing accuracy through longer trajectories.

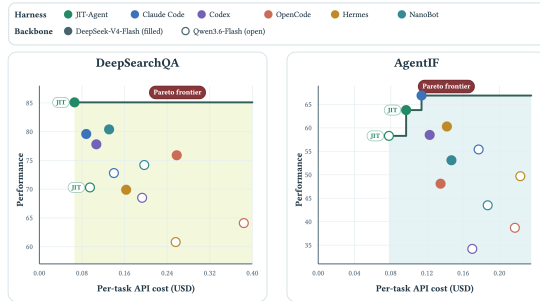


Figure 3: **Cost–performance trade-offs on DeepSearchQA and AgentIF.** Marker color identifies the harness, while filled and open circles distinguish DeepSeek-V4-Flash and Qwen3.6-Flash. The horizontal axis is API cost per case in USD and the vertical axis is task performance. The dark-green step line traces the global Pareto frontier, and the pale yellow-green and blue-green regions contain pairings dominated by at least one Pareto-optimal point.

Palimpsest – graph-planned artifact execution

A task becomes a dependency graph, executes with bounded parallelism, and leaves reusable state.

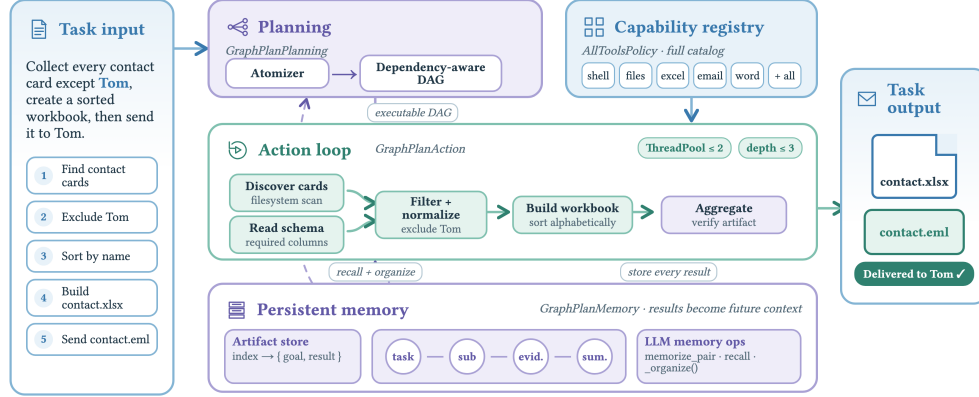


Figure 5: **Palimpsest: graph-planned artifact execution.** GraphPlanPlanning converts the contact-processing request into a DAG; GraphPlanAction executes it with bounded width and depth, while GraphPlanMemory stores reusable artifacts and reasoning state.

5.5 GENERALIZATION ACROSS MODEL PAIRS

To test transfer across model families and scales, we compare ReAct with JIT-generated harnesses on six fixed backbones from DeepSeek V4, Qwen3.6, and MIMO-V2.5 across four benchmarks. Figure 4 shows that JIT-Agent wins all 24 matched comparisons by 7.6 points on average, with family-level gains of 10.2, 4.0, and 8.6 points, respectively. DeepSearchQA benefits most (+15.2 on average), including +22.2 for MIMO-V2.5-Pro and +19.0 for DeepSeek-V4-Flash, while DeepPlanning-Shopping reaches a +24.8 gain. These consistent within-backbone improvements show that harness intelligence transfers across model families and variants rather than compensating for one backbone.

5.6 VISUALIZATION OF GENERATED HARNESSES

To inspect the generated structures, Figure 5 presents PALIMPSEST, a harness for discovering contact cards, filtering and normalizing records, constructing a workbook, and delivering it by email. It compiles these dependencies into a DAG: discovery precedes filtering, workbook construction waits for normalized records, and delivery follows artifact verification. GraphPlanAction executes ready nodes with bounded width and depth, while GraphPlanMemory preserves goals, evidence, and artifacts for downstream nodes. This task-shaped coordination of planning, memory, and execution goes beyond prompt variation and turns the request into a verifiable production pipeline; Section C shows complementary structures for recursive delegation, context folding, gated tools, phase-conditioned execution, evidence tracking, numerical state, and workspace repair.

6 CONCLUSION

We introduced JIT-Agent, a dedicated model for synthesizing, repairing, and evolving task-conditioned agent harnesses at inference time. By expressing the harness through a composable four-module protocol and training the generator through customization learning, repair supervision, and Evo-GDPO, JIT-Agent turns harness construction from a manually engineered, ahead-of-time artifact into a learned capability. Across deep research, daily work, planning, and workspace tasks, JIT-generated harnesses consistently strengthen their underlying backbones, compete with frontier models and advanced fixed harnesses, and improve cost–performance trade-offs. These results establish harness intelligence, *i.e.*, the capacity to adapt the operational scaffold through which a model acts, as a trainable and transferable source of agent capability beyond model weights alone.

CONTRIBUTORS

Core Contributors: Guibin Zhang, Leo Lu, Fangzhou Xie

Contributors: Kang Zhu, Junhao Wang, Zhifei Xie, Zhaochen Yu, Zihang Liu, Zhongxiang Sun, Qiankun Li, Yue Liao, Heng Chang, Xiaobin Hu, Qibing Ren

Corresponding: Wangchunshu Zhou, Chuanrui Hu, Yafeng Deng, and Shuicheng Yan

Affiliations: National University of Singapore, EverMind AI, Nanyang Technological University

REFERENCES

Salaheddin Alzu’bi, Baran Nama, Arda Kaz, Anushri Eswaran, Weiyuan Chen, Sarvesh Khetan, Rishab Bala, Tu Vu, and Sewoong Oh. Roma: Recursive open meta-agent framework for long-horizon multi-agent systems, 2026. URL <https://arxiv.org/abs/2602.01848>.

Anthropics. GitHub - anthropics/claude-code: Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands. — github.com. <https://github.com/anthropics/claude-code>, 2025. [Accessed 04-01-2026].

Apodex Team. Apodex-1.0: Self-evolving heavy-duty solver. <https://www.apodex.com/blog/apodex-1.0>, 2026.

Fei Bai, Huatong Song, Shuang Sun, Daixuan Cheng, Yike Yang, Chuan Hao, Renyuan Li, Feng Chang, Yuan Wei, Ran Tao, Bryan Dai, Jian Yang, Wayne Xin Zhao, and Ji-Rong Wen. Clawgym: A scalable framework for building effective claw agents, 2026. URL <https://arxiv.org/abs/2604.26904>.

Pengfei Cao, Tianyi Men, Wencan Liu, Jingwen Zhang, Xuzhao Li, Xixun Lin, Dianbo Sui, Yanan Cao, Kang Liu, and Jun Zhao. Large language models for planning: A comprehensive and systematic survey, 2025. URL <https://arxiv.org/abs/2505.19683>.

Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, Chen Sun, Han Hou, Hui Yang, James Pan, Jianan Lou, Jiayi Mao, Jizheng Liu, Jinpeng Li, Kangyi Liu, Kenkun Liu, Rui Wang, Run Li, Tong Niu, Wenlong Zhang, Wenqi Yan, Xuancheng Wang, Yuchen Zhang, Yi-Hsin Hung, Yuan Jiang, Zexuan Liu, Zihan Yin, Zijian Ma, and Zhiwen Mo. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations, 2025a. URL <https://arxiv.org/abs/2506.13651>.

Kaiyuan Chen, Qimin Wu, Taiyu Hou, Tianhao Tang, Xueyu Hu, Yuchen Hou, Bikun Li, Chengming Qian, Guoyin Wang, Haolin Chen, Haotong Tian, Haoye Zhang, Haoyu Bian, Hongbing Pan, Hongkang Zhang, Hongyi Zhou, Jiaqi Cai, Jiewu Rao, Jiyuan Ren, Keduan Huang, Lucia Zhu Huang, Mingyu Yuan, Naixu Guo, Qicheng Tang, Qinyan Zhang, Shuai Chen, Siheng Chen, Ting Ting Li, Xiaoxing Guo, Yaocheng Zuo, Yaoqi Guo, Yinan Wang, Yinzhou Yu, Yize Wang, Yuan Jiang, Yuan Tian, Yuanshuo Zhang, Yuxuan Liu, Yvette Yan Zeng, Zhenyu Shan, Zihan Yin, Xiaobo Hu, Yang Liu, Yixin Ren, and Yuan Gong. Agentif-oneday: A task-level instruction-following benchmark for general ai agents in daily scenarios, 2026a. URL <https://arxiv.org/abs/2601.20613>.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, Yuan Xie, Heng Qu, Kun Shao, and Jian Luan. Harnessx: A composable, adaptive, and evolvable agent harness foundry, 2026b. URL <https://arxiv.org/abs/2606.14249>.

Zijian Chen, Xueguang Ma, Shengyao Zhuang, Ping Nie, Kai Zou, Andrew Liu, Joshua Green, Kshama Patel, Ruoxi Meng, Mingyi Su, Sahel Sharifymoghaddam, Yanxi Li, Haoran Hong, Xinyu Shi, Xuye Liu, Nandan Thakur, Crystina Zhang, Luyu Gao, Wenhui Chen, and Jimmy Lin. Browsecmp-plus: A more fair and transparent evaluation benchmark of deep-research agent, 2025b. URL <https://arxiv.org/abs/2508.06600>.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.

ClaudeCode. Agent Skills - Claude Code Docs — code.claude.com. <https://code.claude.com/docs/en/skills>, 2025. [Accessed 03-01-2026].

- DeepSeek-AI. Deepseek-v4 preview release. <https://api-docs.deepseek.com/news/news260424/>, 2026.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks. *arXiv preprint arXiv:2503.09572*, 2025a.
- Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks, 2025b. URL <https://arxiv.org/abs/2503.09572>.
- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro>, 2026a.
- Google DeepMind. Gemini 3.5 flash model card. <https://deepmind.google/models/model-cards/gemini-3-5-flash/>, 2026b.
- Nikita Gupta, Riju Chatterjee, Lukas Haas, Connie Tao, Andrew Wang, Chang Liu, Hidekazu Oiwa, Elena Gribovskaya, Jan Ackermann, John Blitzer, Sasha Goldshtein, and Dipanjan Das. Deepsearchqa: Bridging the comprehensiveness gap for deep research agents, 2026. URL <https://arxiv.org/abs/2601.20975>.
- HKUDS. Nanobot: Lightweight open-source ai agent. <https://github.com/HKUDS/nanobot>, 2026.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. Evermemos: A self-organizing memory operating system for structured long-horizon reasoning, 2026a. URL <https://arxiv.org/abs/2601.02163>.
- Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language model, 2024a. URL <https://arxiv.org/abs/2408.09559>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024b.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2025. URL <https://arxiv.org/abs/2512.13564>.
- Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. Memory in the age of ai agents, 2026b. URL <https://arxiv.org/abs/2512.13564>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*, 2023.
- LangGraph. GitHub - langchain-ai/langgraph: Build resilient language agents as graphs. — github.com. <https://github.com/langchain-ai/langgraph>, 2025. [Accessed 03-01-2026].
- Hyunin Lee, Jinglue Xu, Jeffrey Seely, Donghyun Lee, Matei Zaharia, and Yujin Tang. Recursive harness self-improvement, 2026a. URL <https://arxiv.org/abs/2607.15524>.

- Sanghoon Lee, Jiyeong Chae, and Kyung-Joon Park. Harness engineering for physical ai: Robot middleware is the harness layer, 2026b. URL <https://arxiv.org/abs/2606.09416>.
- Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses, 2026c. URL <https://arxiv.org/abs/2603.28052>.
- Yoonsang Lee, Howard Yen, Xi Ye, and Danqi Chen. Agentic aggregation for parallel scaling of long-horizon agentic tasks, 2026d. URL <https://arxiv.org/abs/2604.11753>.
- Xiaoxi Li, Wenxiang Jiao, Jiarui Jin, Guanting Dong, Jiajie Jin, Yinuo Wang, Hao Wang, Yutao Zhu, Ji-Rong Wen, Yuan Lu, and Zhicheng Dou. Deepagent: A general reasoning agent with scalable toolsets, 2026. URL <https://arxiv.org/abs/2510.21618>.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses, 2026. URL <https://arxiv.org/abs/2604.25850>.
- Shih-Yang Liu, Xin Dong, Ximing Lu, Shizhe Diao, Peter Belcak, Mingjie Liu, Min-Hung Chen, Hongxu Yin, Yu-Chiang Frank Wang, Kwang-Ting Cheng, Yejin Choi, Jan Kautz, and Pavlo Molchanov. Gdpo: Group reward-decoupled normalization policy optimization for multi-reward rl optimization, 2026a. URL <https://arxiv.org/abs/2601.05242>.
- Zewen Liu, Zhan Shi, Yisi Sang, Bing He, Minhua Lin, Tianxin Wei, Dakuo Wang, Benoit Dumoulin, Wei Jin, and Hanqing Lu. Adaptive auto-harness: Sustained self-improvement for agentic system deployment on open-ended task streams, 2026b. URL <https://arxiv.org/abs/2606.01770>.
- Xinghua Lou, Miguel Lázaro-Gredilla, Antoine Dedieu, Carter Wendelken, Wolfgang Lehrach, and Kevin P. Murphy. AutoHarness: Improving llm agents by automatically synthesizing a code harness, 2026. URL <https://arxiv.org/abs/2603.03329>.
- Junyu Luo, Weizhi Zhang, Ye Yuan, Yusheng Zhao, Junwei Yang, Yiyang Gu, Bohan Wu, Binqi Chen, Ziyue Qiao, Qingqing Long, Rongcheng Tu, Xiao Luo, Wei Ju, Zhiping Xiao, Yifan Wang, Meng Xiao, Chenwu Liu, Jingyang Yuan, Shichang Zhang, Yiqiao Jin, Fan Zhang, Xian Wu, Hanqing Zhao, Dacheng Tao, Philip S. Yu, and Ming Zhang. Large language model agent: A survey on methodology, applications and challenges, 2025. URL <https://arxiv.org/abs/2503.21460>.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. *Preprints*, April 2026a. doi: 10.20944/preprints202604.0428.v2. URL <https://doi.org/10.20944/preprints202604.0428.v2>.
- Qianyu Meng, Yanan Wang, Liyi Chen, Qimeng Wang, Chengqiang Lu, Wei Wu, Yan Gao, Yi Wu, and Yao Hu. Agent harness for large language model agents: A survey. 2026b.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL <https://arxiv.org/abs/2601.11868>.
- Microsoft. GitHub - ag2ai/ag2: AG2 (formerly AutoGen): The Open-Source AgentOS. <https://github.com/ag2ai/ag2>, 2024. [Accessed 31-08-2025].

- Moonshot AI. Kimi k2.7 code: Open-source agentic coding model. <https://www.kimi.com/resources/kimi-k2-7-code>, 2026.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Yike Guo, Xinmei Tian, and Bo Han. TTHE: Test-time harness evolution, 2026. URL <https://arxiv.org/abs/2607.08124>.
- Xuying Ning, Katherine Tieu, Dongqi Fu, Tianxin Wei, Zihao Li, Yuanchen Bei, Jiaru Zou, Mengting Ai, Zhining Liu, Ting-Wei Li, et al. Code as agent harness. *arXiv preprint arXiv:2605.18747*, 2026.
- Nous Research. Hermes agent. <https://github.com/NousResearch/hermes-agent>, 2026.
- OpenAI. Codex | AI Coding Partner from OpenAI — openai.com. <https://openai.com/codex/>, 2025.
- OpenAI. Gpt-5.6: Frontier intelligence that scales with your ambition. <https://openai.com/index/gpt-5-6/>, 2026.
- OpenClaw Contributors. Openclaw: Your own personal ai assistant. <https://github.com/openclaw/openclaw>, 2026.
- OpenCode-AI. GitHub - opencode-ai/opencode: A powerful AI coding agent. Built for the terminal. — github.com. <https://github.com/opencode-ai/opencode>, 2025.
- Linyue Pan, Lexiao Zou, Shuo Guo, Jingchen Ni, and Hai-Tao Zheng. Natural-language agent harnesses. *arXiv preprint arXiv:2603.25723*, 2026.
- PinchBench. Best Models by Success Rate | PinchBench — pinchbench.com. <https://pinchbench.com/>, 2026. [Accessed 11-04-2026].
- Hongjin Qian, Zhao Cao, and Zheng Liu. Memobrain: Executive memory as an agentic brain for reasoning, 2026. URL <https://arxiv.org/abs/2601.08079>.
- Tianrui Qin, Qianben Chen, Sinuo Wang, He Xing, King Zhu, He Zhu, Dingfeng Shi, Xinxin Liu, Ge Zhang, Jiaheng Liu, Yuchen Eleanor Jiang, Xitong Gao, and Wangchunshu Zhou. Flash-searcher: Fast and effective web agents via dag-based parallel execution, 2025. URL <https://arxiv.org/abs/2509.25301>.
- Qwen Team. Qwen3.7-plus: Multimodal agent intelligence. <https://qwen.ai/home>, 2026.
- Jianhao Ruan, Zhihao Xu, Yiran Peng, Fashen Ren, Zhaoyang Yu, Xinbing Liang, Jinyu Xiang, Yongru Chen, Bang Liu, Chenglin Wu, Yuyu Luo, and Jiayi Zhang. Aorchestra: Automating sub-agent creation for agentic orchestration, 2026. URL <https://arxiv.org/abs/2602.03786>.
- Anjana Sarkar and Soumyendu Sarkar. Survey of llm agent communication with mcp: A software design pattern centric review, 2025. URL <https://arxiv.org/abs/2506.05364>.
- Yu Shang, Yu Li, Keyu Zhao, Likai Ma, Jiahe Liu, Fengli Xu, and Yong Li. Agentsquare: Automatic llm agent search in modular design space. *arXiv preprint arXiv:2410.06153*, 2024.
- Shuai Shao, Kangning Zhang, Qingyao Li, Shijian Wang, Hao Wang, Wenxiang Jiao, Yuan Lu, Yi Guo, Weiwen Liu, and Weinan Zhang. Harness-R1: Learning to edit executable runtime harnesses from agent failure trajectories, 2026. URL <https://arxiv.org/abs/2608.02276>.
- Dingfeng Shi, Jingyi Cao, Qianben Chen, Weichen Sun, Weizhen Li, Hongxuan Lu, Fangchen Dong, Tianrui Qin, King Zhu, Minghao Liu, Jian Yang, Ge Zhang, Jiaheng Liu, Changwang Zhang, Jun Wang, Yuchen Eleanor Jiang, and Wangchunshu Zhou. Taskcraft: Automated generation of agentic tasks, 2025. URL <https://arxiv.org/abs/2506.10055>.
- Xiaoshuai Song, Haoifei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. Envscaler: Scaling tool-interactive environments for llm agent via programmatic synthesis, 2026. URL <https://arxiv.org/abs/2601.05808>.
- Weixuan Wang, Dongge Han, Daniel Madrigal Diaz, Jin Xu, Victor Rühle, and Saravan Rajmohan. Odysseybench: Evaluating llm agents on long-horizon complex office application workflows, 2025. URL <https://arxiv.org/abs/2508.09124>.

- Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. Officebench: Benchmarking language agents across multiple applications for office automation, 2024. URL <https://arxiv.org/abs/2407.19056>.
- Jialong Wu, Wenbiao Yin, Yong Jiang, Zhenglin Wang, Zekun Xi, Runnan Fang, Linhai Zhang, Yulan He, Deyu Zhou, Pengjun Xie, and Fei Huang. Webwalker: Benchmarking llms in web traversal, 2025. URL <https://arxiv.org/abs/2501.07572>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkang Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation framework, August 01, 2023 2023.
- Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization, 2026. URL <https://arxiv.org/abs/2509.13313>.
- Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward, 2026. URL <https://arxiv.org/abs/2602.12430>.
- B. Y. Yan, Chaofan Li, Hongjin Qian, Shuqi Lu, and Zheng Liu. General agentic memory via deep research, 2025. URL <https://arxiv.org/abs/2511.18423>.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024. URL <https://arxiv.org/abs/2405.15793>.
- Xiaofang Yang, Lijun Li, Heng Zhou, Tong Zhu, Xiaoye Qu, Yuchen Fan, Qianshan Wei, Rui Ye, Li Kang, Yiran Qin, Zhiqiang Kou, Daizong Liu, Qi Li, Ning Ding, Siheng Chen, and Jing Shao. Toward efficient agents: Memory, tool learning, and planning, 2026. URL <https://arxiv.org/abs/2601.14192>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023a. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, 2023b.
- Rui Ye, Zhongwang Zhang, Kuan Li, Huifeng Yin, Zhengwei Tao, Yida Zhao, Liangcai Su, Liwen Zhang, Zile Qiao, Xinyu Wang, Pengjun Xie, Fei Huang, Siheng Chen, Jingren Zhou, and Yong Jiang. Agentfold: Long-horizon web agents with proactive context management, 2025. URL <https://arxiv.org/abs/2510.24699>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Z.ai. Glm-5.2: Built for long-horizon tasks. <https://z.ai/blog/glm-5.2>, 2026.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026a. URL <https://arxiv.org/abs/2512.24601>.
- Guibin Zhang, Haotian Ren, Chong Zhan, Zhenhong Zhou, Junhao Wang, He Zhu, Wangchunshu Zhou, and Shuicheng Yan. Memevolve: Meta-evolution of agent memory systems, 2025. URL <https://arxiv.org/abs/2512.18746>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating Agentic Workflow Generation, October 2024. URL <http://arxiv.org/abs/2410.10762>. arXiv:2410.10762.

- Yinger Zhang, Shutong Jiang, Renhao Li, Jianhong Tu, Yang Su, Lianghao Deng, Xudong Guo, Chenxu Lv, and Junyang Lin. Deepplanning: Benchmarking long-horizon agentic planning with verifiable constraints, 2026b. URL <https://arxiv.org/abs/2601.18137>.
- Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Chunyang Zhu, Jiaying Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Zhengru Fang, Yi Nie, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness vla: Steering frozen vlas into reliable manipulation primitives via memory-guided agents, 2026c. URL <https://arxiv.org/abs/2607.08448>.
- Chenyu Zhou, Huacan Chai, Wenteng Chen, Zihan Guo, Rong Shan, Yuanyi Song, Tianyi Xu, Yingxuan Yang, Aofan Yu, Weiming Zhang, Congming Zheng, Jiachen Zhu, Zeyu Zheng, Zhuosheng Zhang, Xingyu Lou, Changwang Zhang, Zhihui Fu, Jun Wang, Weiwen Liu, Jianghao Lin, and Weinan Zhang. Externalization in llm agents: A unified review of memory, skills, protocols and harness engineering, 2026. URL <https://arxiv.org/abs/2604.08224>.
- Tailin Zhou. Hierarchical self-improvement: A framework for task-specific evolvable agent harnesses, 2026. URL <https://arxiv.org/abs/2608.08466>.
- He Zhu, Tianrui Qin, King Zhu, Heyuan Huang, Yeyi Guan, Jinxiang Xia, Yi Yao, Hanhao Li, Ningning Wang, Pai Liu, Tianhao Peng, Xin Gui, Xiaowan Li, Yuhui Liu, Yuchen Eleanor Jiang, Jun Wang, Changwang Zhang, Xiangru Tang, Ge Zhang, Jian Yang, Minghao Liu, Xitong Gao, Jiaheng Liu, and Wangchunshu Zhou. Oagents: An empirical study of building effective agents, 2025. URL <https://arxiv.org/abs/2506.15741>.

A HARNESS OPTIMIZATION PARADIGMS

Table 4 organizes representative harness-optimization methods by when the harness is constructed and which parts of improvement are learned. Existing approaches primarily search for a durable harness ahead of deployment or edit that artifact online, whereas JIT-Agent amortizes construction into a trained generator that synthesizes an instance-specific harness and learns both repair and evolution. The comparison is structural rather than a ranking of empirical performance: it clarifies which capabilities are native to the optimization procedure and which remain external search operations.

Table 4: **Harness optimization paradigms.** *Construction* distinguishes harnesses found by ahead-of-time search, ahead-of-time harnesses subsequently edited with test-time feedback, and harnesses generated just in time for each task. *Instance synthesis*, *Harness model*, *Learned repair*, and *Online evolution* indicate whether a method directly synthesizes an instance-specific harness, trains the generator, learns repair from failed execution trajectories, and continues improving after deployment, respectively.

Method	Construction	Instance synthesis	Harness model	Learned repair	Online evolution
AutoHarness (Lou et al., 2026)	AOT (search)	✗	✗	✗	✗
Meta-Harness (Lee et al., 2026c)	AOT (search)	✗	✗	✗	✗
AHE (Lin et al., 2026)	AOT (search)	✗	✗	✗	✗
Adaptive AH (Liu et al., 2026b)	AOT (test-time editing)	✗	✗	✗	✓
TTHE (Nie et al., 2026)	AOT (test-time editing)	✗	✗	✗	✓
RHI (Lee et al., 2026a)	AOT (test-time editing)	✗	✗	✗	✓
Harness-R1 (Shao et al., 2026)	AOT (test-time editing)	✗	✓	✓	✓
JIT-Agent (ours)	JIT	✓	✓	✓	✓

B ADDITIONAL RESULTS

This section complements the aggregate comparisons in the main paper with the full streaming evaluation, which tests whether completed tasks can improve the harness bank for later tasks without changing the generator parameters.

B.1 TEST-TIME HARNESS EVOLUTION

JIT-Agent is designed not only to generate a harness once, but also to improve its archive from execution feedback. We compare *Static JIT*, where harness generations are independent, with *Streaming JIT*, which retrieves and updates harnesses throughout the evaluation stream.

Figure 6 shows that Streaming JIT finishes above the static variant on DeepPlanning-Shopping, DeepPlanning-Travel, and OfficeBench. Across all three streams, the cumulative-accuracy advantage emerges as execution feedback accumulates and remains positive through the end of evaluation. The accompanying cost and tool-call traces suggest that the endpoint gains are not uniformly coupled to larger interaction budgets. This pattern is consistent with the objective of Evo-GDPO, which retains harnesses only when they advance the archive frontier.

C ADDITIONAL GENERATED HARNESS VISUALIZATIONS

This appendix complements the representative case in Section 5.6 with nine additional harnesses generated under the same four-module protocol. The examples span bounded recursive research, constraint-heavy shopping and travel, phased web production, clue-driven research, evidence-sensitive retrieval, numerical analysis, and workspace manipulation. They are not variants of a single execution template: JIT-Agent selects a state representation and control regime suited to the structure of each request, then instantiates planning, action, memory, and capability orchestration around that choice. Collectively, the cases make task-conditioned harness customization visible at the level of execution semantics rather than prompt wording.

Deep research as bounded recursive delegation. TRAPDOOR addresses a multi-hop identity question whose clues span a historical quotation, a university merger, a memoir, and a research paper. Here a fixed DAG would commit too early to an uncertain evidence path, so the generated harness uses `DynamicDecomposer` to form and revise sub-questions and augments the research tool policy with a synthetic `delegate` capability. A `delegate` call opens a private research subagent with its

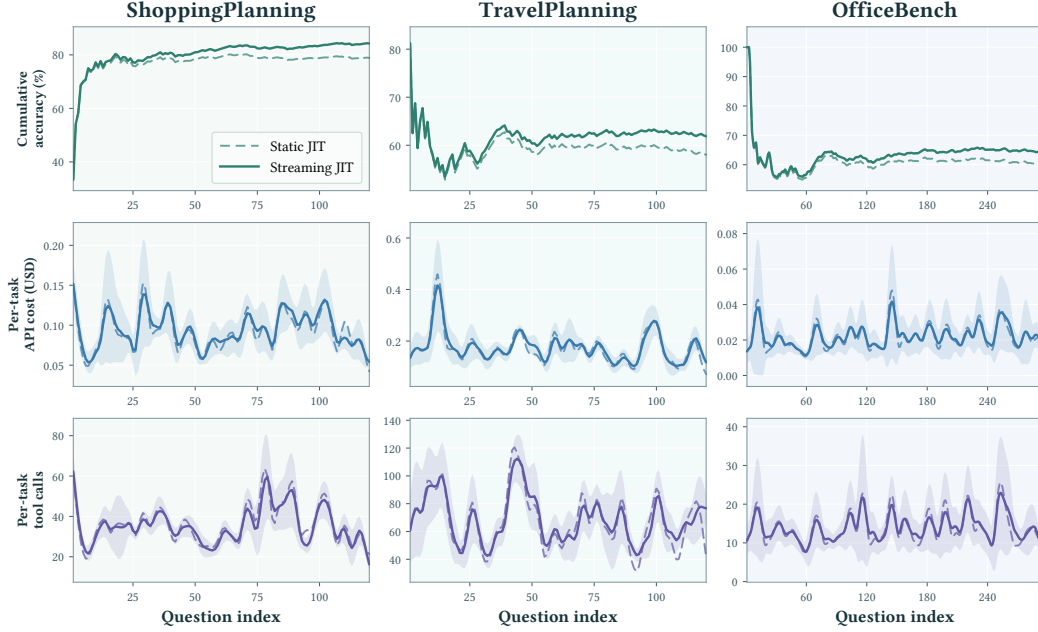


Figure 6: **Streaming test-time harness evolution across task streams.** Cumulative accuracy (top), per-task API cost (middle), and per-task tool calls (bottom) on DeepPlanning-Shopping, DeepPlanning-Travel, and OfficeBench. Dashed curves represent Static JIT, where task-specific harness generations are independent, while solid curves represent Streaming JIT, which continuously incorporates execution feedback as new tasks arrive. Shaded regions show local variation around the streaming trajectories. Streaming JIT finishes with higher cumulative accuracy on all three benchmarks, while API-cost and tool-use trajectories remain task-dependent and of broadly similar scale.

JIT HARNESS ATLAS · 02

Trapdoor — delegation as a runtime primitive

A synthetic tool call opens a bounded sub-agent; its answer returns through the ordinary observation loop.

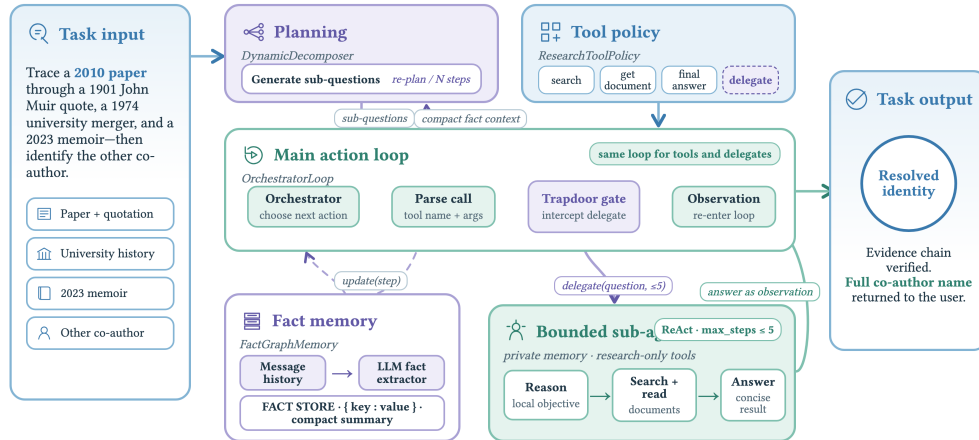


Figure C.1: **Trapdoor: bounded research behind a delegated tool call.** A synthesized delegate capability is intercepted by OrchestratorLoop, which runs a bounded subagent and writes extracted facts into FactGraphMemory.

own memory, research-only tools, and a five-step budget; the returned answer re-enters the parent loop as an ordinary observation. In parallel, FactGraphMemory extracts compact key–value facts from the growing history, allowing the parent to integrate evidence across branches without inheriting every subagent trace. Thus, the harness makes recursive research a runtime primitive specifically for a task whose uncertainty is resolved through branching evidence collection.

JIT HARNESS ATLAS · 03

Origami — hierarchical work, folded context

Independent subtask memories preserve the work; a deliberate fold replaces only the active context.

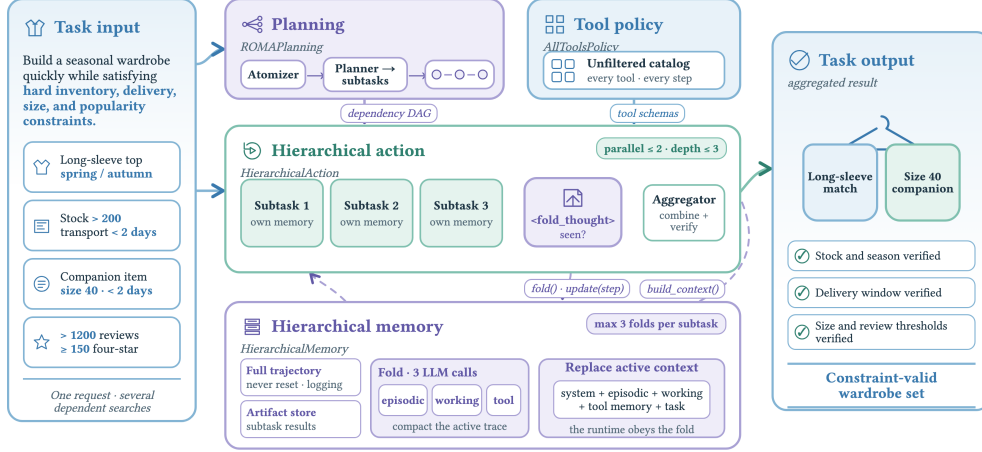


Figure C.2: **Origami: hierarchical work with folded context.** ROMAPPlanning creates isolated subtasks; HierarchicalMemory retains their trajectories and artifacts while fold_thought replaces only the active working context before aggregation.

JIT HARNESS ATLAS · 04

Turnstile — evidence first, itinerary second

A task-specific checklist withholds final_answer until every required travel fact has been collected.

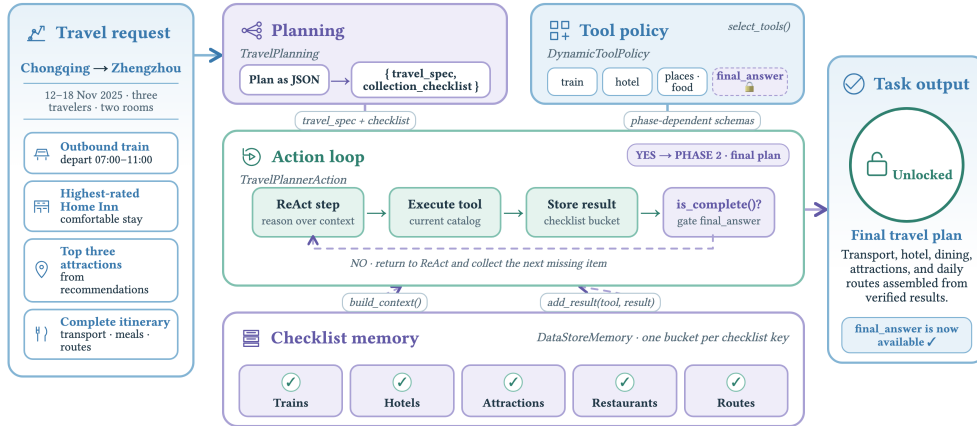


Figure C.3: **Turnstile: evidence first, itinerary second.** TravelPlanning emits a travel specification and checklist, DataStoreMemory tracks the required evidence buckets, and DynamicToolPolicy exposes final_answer only after is_complete() succeeds.

Figures C.2 and C.3 illustrate two task-specific ways to control long-horizon execution through memory. ORIGAMI must assemble a seasonal wardrobe under coupled stock, delivery, size, review-count, and rating constraints. ROMAPanning decomposes the request into dependent product searches, HierarchicalAction runs at most two branches in parallel to bounded depth, and each branch retains its own trajectory and artifacts. When a branch grows long, fold_thought replaces only its active working context; completed subtask results remain available for the final constraint check. The harness therefore spends context on the unresolved portion of a combinatorial shopping task without discarding earlier product evidence.

TURNSTILE, by contrast, serves a travel request that specifies dates, passengers, rooms, a departure window, a highest-rated hotel, attractions, meals, and daily routes. The generated TravelPlanning module compiles these obligations into a typed travel_spec and collection checklist, and DataStoreMemory assigns a dedicated bucket to each evidence class. DynamicToolPolicy exposes search tools for the missing bucket but withholds final_answer until is_complete() succeeds. Here memory is not used primarily to compress a long trace; it acts as an executable coverage contract that prevents itinerary synthesis before the required travel facts have been collected.

JIT HARNESS ATLAS · 05

Gearbox — one phase shift rewires the agent

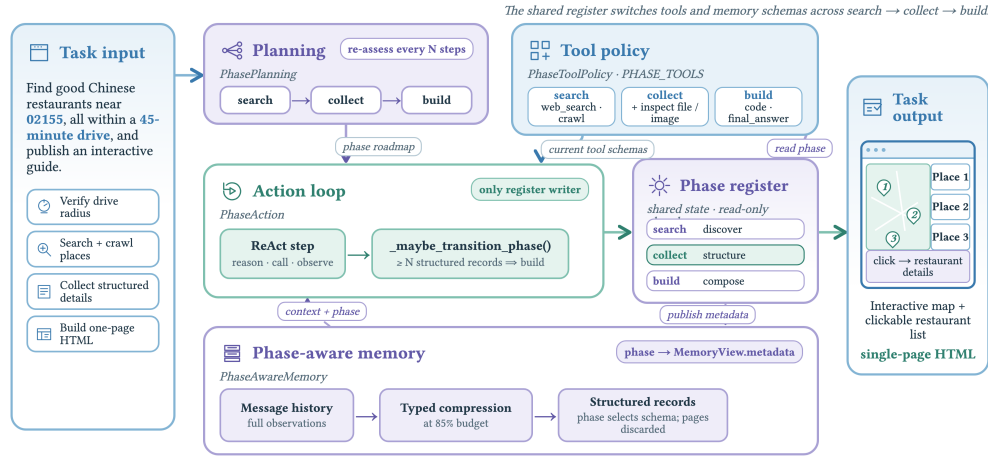


Figure C.4: **Gearbox: one phase shift rewrites the agent.** PhaseAction is the sole writer of a shared phase register; PhaseToolPolicy and PhaseAwareMemory read that state to switch both exposed capabilities and typed memory schemas.

Figures C.4 and C.5 show how explicit state can continuously reshape later decisions. GEARBOX is generated to find Chinese restaurants within a specified driving radius and publish the results as an interactive one-page guide. Its PhaseAction is the sole writer of a shared search–collect–build register; the transition to construction occurs only after enough structured restaurant records have been gathered. Both PhaseToolPolicy and PhaseAwareMemory read that register, switching from web discovery, to record inspection, to code and publication while compressing observations into the schema needed by the current phase. The model thus creates a phase machine because the task changes modality from evidence acquisition to artifact construction.

PEGBBOARD instead tackles an identification problem defined by six biographical and film clues. It represents progress as a candidate × clue matrix in which every extracted claim retains its document identifier. Empty cells and contradictions drive DynamicResearchPlanning toward targeted searches, while a sufficiently supported row triggers a separate verification phase that reopens cited documents and resolves conflicts. Unlike a generic research transcript, this state makes coverage and source provenance directly actionable. The contrast with GEARBOX shows that generated state may

JIT HARNESS ATLAS · 06

Pegboard — research as an evidence matrix

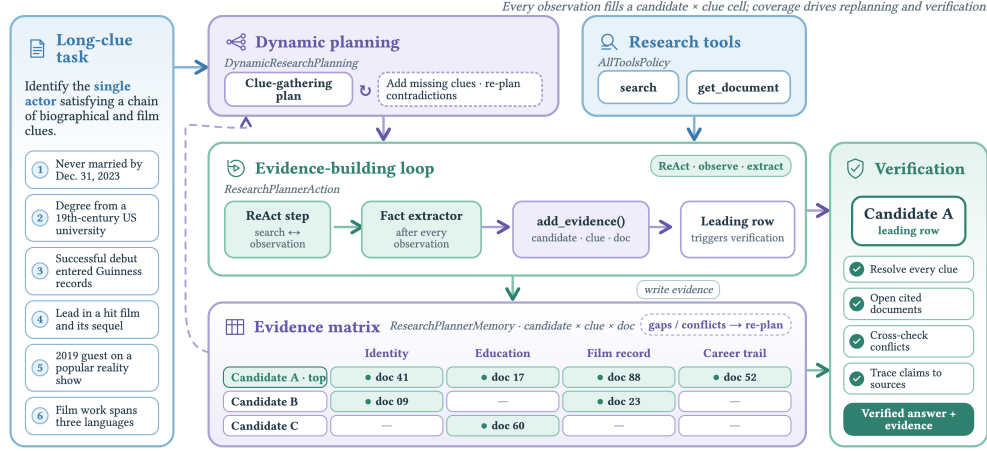


Figure C.5: **Pegboard: research as an evidence matrix.** Every observation is extracted into a candidate × clue cell with a document identifier; matrix coverage drives both DynamicResearchPlanning and the transition to source-grounded verification.

be a small phase variable or a task-shaped evidence structure, depending on what must govern the next action.

JIT HARNESS ATLAS · 07

Appraiser — evidence decides what the model sees

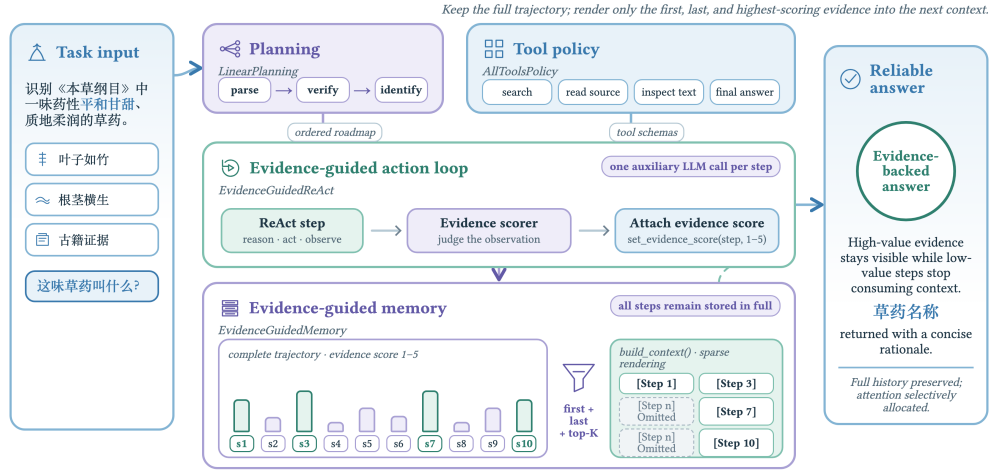


Figure C.6: **Appraiser: evidence decides what the model sees.** EvidenceGuidedReAct scores every step, while EvidenceGuidedMemory stores the full run but renders only the first, last, and top- K observations into build_context().

Figures C.6 and C.7 customize what subsequent reasoning receives from prior execution. APPRAISER answers a classical materia-medica identification question by reconciling botanical traits with textual evidence. After every search or reading step, an auxiliary scorer assigns the observation an evidence

Abacus — from computation to reliable typed state

Shell-derived evidence is parsed once, preserved structurally, and reported without re-deriving the numbers.

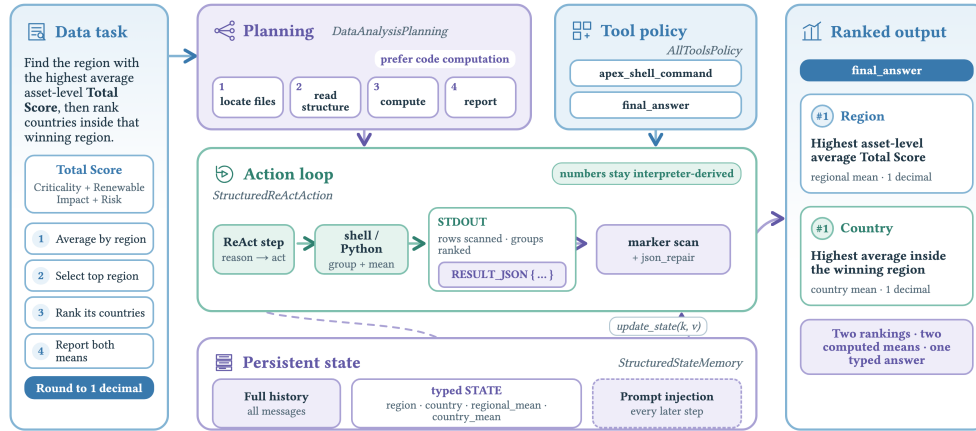


Figure C.7: **Abacus: from computation to reliable typed state.** StructuredReActAction executes shell or Python code, extracts a RESULT_JSON payload from standard output, and updates StructuredStateMemory without asking the model to re-derive computed values.

value; EvidenceGuidedMemory keeps the complete trajectory for auditability but renders only the first, last, and top- K observations into the next context. Low-value exploration therefore stops competing with the passages most useful for identification, while no evidence is erased from persistent history. This harness is tailored to a retrieval task in which the main bottleneck is evidence salience rather than tool choice.

ABACUS is generated for a numerical analysis request that first compares regional averages and then ranks countries within the winning region. The harness deliberately moves aggregation out of language-model reasoning: StructuredReActAction invokes shell or Python, extracts a marked RESULT_JSON payload from standard output, repairs its syntax if necessary, and writes the values into StructuredStateMemory. Region, country, regional mean, and country mean are then injected as typed state into every later step, so the final answer reports interpreter-derived quantities instead of re-deriving them from prose. Both cases preserve a full audit trail, but each renders the working view around its task's dominant reliability requirement—evidence selection for one and numerical fidelity for the other.

Figures C.8 and C.9 specialize workspace execution at different scales by controlling when actions enter persistent history. PLAYER PIANO handles a batch migration of configuration files, including database, logging, and API-endpoint changes. StructuredPlanning emits typed *discover*, *process-file*, and *final-answer* steps that a Python dispatcher executes; after each edit, a deterministic non-LLM check decides whether to advance or retry. ProgressiveTaskMemory renders a per-file board into the system prompt and retains both originals and accepted changes, preventing a large batch from losing track of which files have actually passed verification.

MULLIGAN is generated for a shorter document task: extract a student identifier from a DOCX file and rename that same file. Its linear plan needs no hierarchy or context compression, so the harness retains full successful history and exposes only Word and file operations while hiding unrelated email, spreadsheet, and PDF tools. If an action fails, the loop appends the error locally and regenerates the same step for at most two attempts; only a successful action is committed before the plan advances. The two harnesses therefore share a concern for reliable workspace mutation but realize it differently: one uses typed batch progress and per-file verification, whereas the other uses narrow tool exposure and transactional step repair. This task-dependent difference is precisely the behavior sought from

JIT HARNESS ATLAS · 09

Player Piano — typed batch editing, verified file by file

A Python interpreter executes a structured plan; deterministic checks control every transition.

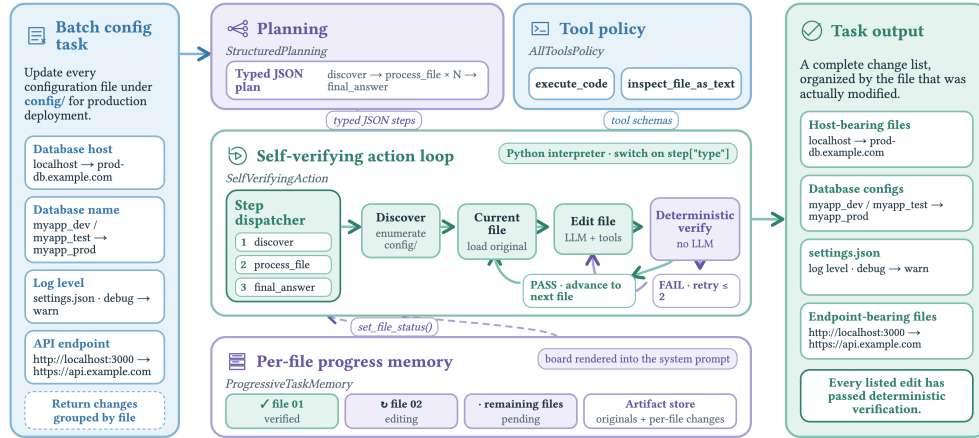


Figure C.8: **Player Piano: typed batch editing, verified file by file.** A Python dispatcher interprets StructuredPlanning steps, SelfVerifyingAction checks each edit without an LLM, and ProgressiveTaskMemory maintains a per-file board and artifact store.

JIT HARNESS ATLAS · 10

Mulligan — repair the step, preserve the history

A failed action is regenerated in place; only successful repairs enter the transcript.

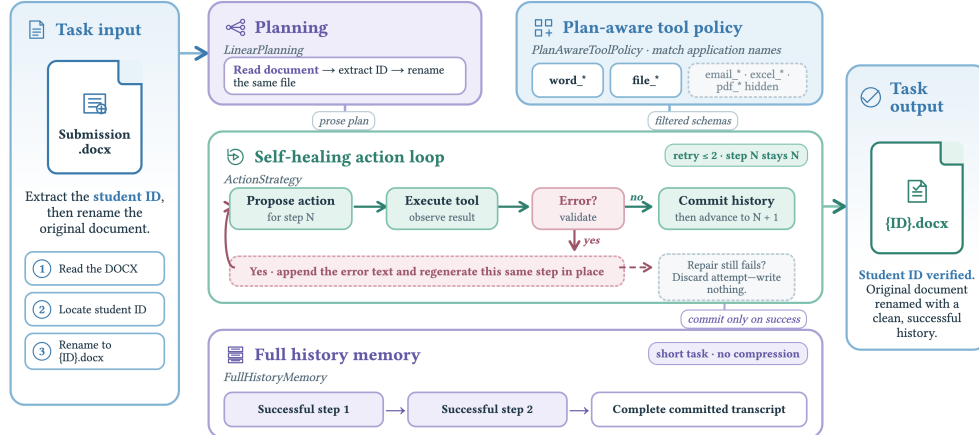


Figure C.9: **Mulligan: repair the step, preserve the history.** PlanAwareToolPolicy exposes only plan-relevant file and document tools; failed actions are regenerated at the same step, and FullHistoryMemory receives only the successful execution.

JIT-Agent: reliability mechanisms are generated to fit the structure and failure surface of the request rather than imposed as one universal scaffold.